Week 2 · Rules and decisions

Programming for Information Systems

Who chooses the rule, and how does a program apply it?

Class warm-up · Research 1 exchange

In groups of three, share the information system you researched.

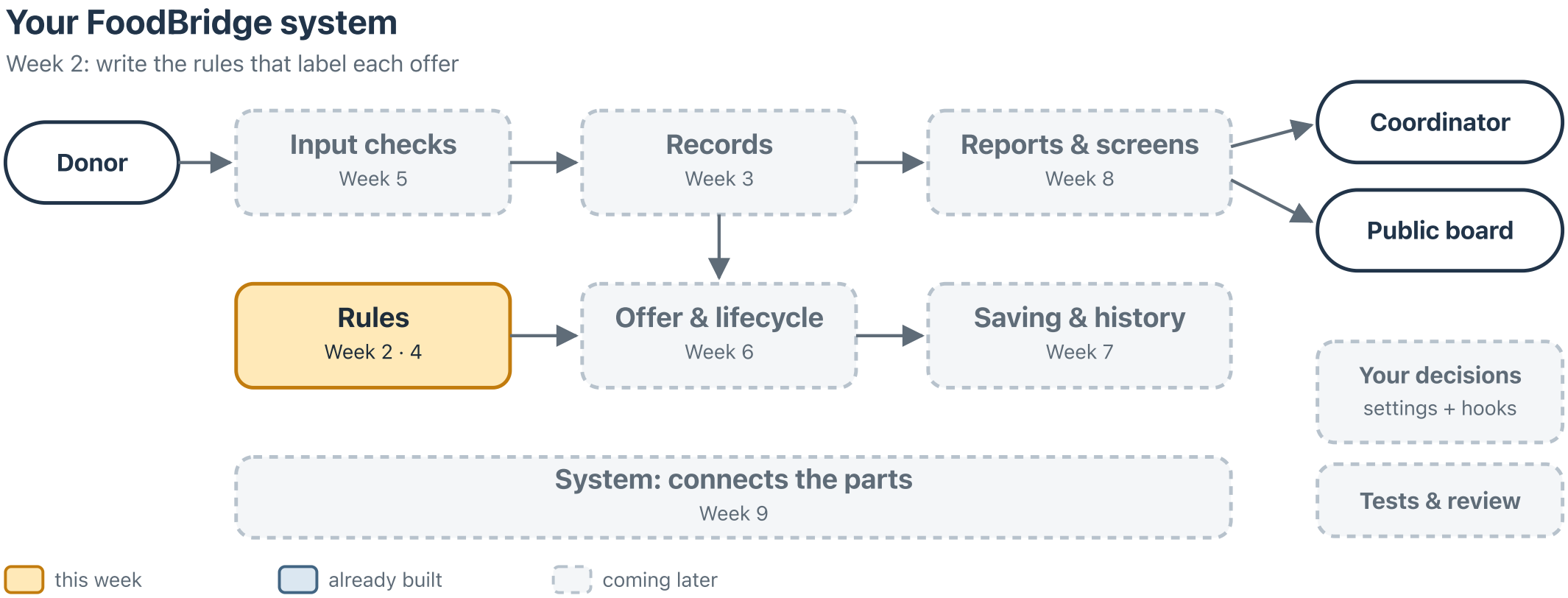
1. Who uses it, and what data does it keep?
2. What rules did you find? Is the rule written down, or only in people's heads?
3. What could go wrong, and who would be affected?

> 📝 Write down one idea from a classmate that you want to remember.

This week

Question	How does a system make a decision, and who chooses the rule?
Python	comparisons, `True` / `False`, `and` / `or` / `not`, `if` / `elif` / `else`, a first small function
Principle 2	Explicit rules: name your numbers, cover every case, test the boundary
You build	`pickup_label` and `urgency`

Your system, by Week 10



The idea: business rules

A **business rule** is a decision about how work is done.

- "*An exam mark of 50 or more is a pass.*"
- "*Children under 6 travel free.*"

> **Coordinator:** "Not approved yet → **hold**. Approved and **20 portions or more** → **team**. Otherwise → **standard**."

- People choose the rule. The program applies it the same way, every time.
- A wrong rule is applied wrongly every time, so rules must be **written exactly** and **tested at the edges**.

Python foundations

How a program decides

Comparisons give `True` or `False`

```
portions = 20
print(portions >= 20)       # True
print(portions > 20)        # False
print(portions == 20)       # True
print("bread" == "Bread")   # False
```

- `==` `!=` `<` `<=` `>` `>=`
- "20 or more" is `>= 20`. "More than 20" is `> 20`.
- Text comparison is exact: capital letters matter.

Combining conditions: `and`, `or`, `not`

```
approved = True
portions = 25
print(approved and portions >= 20)       # both must be True
print(not approved)                      # the opposite
print(portions >= 20 or portions < 0)    # at least one True
```

- `and`: every part must be `True`.
- `or`: at least one part must be `True`.
- `not`: turns `True` into `False`, and the other way.

`if` and `else`: choosing what to run

```
temperature = 31
if temperature >= 30:
    print("Hot: keep the food in the shade")
else:
    print("Normal temperature")
print("This line always runs")
```

- The `if` line ends with a colon `:`.
- The lines that belong to it are **indented** by 4 spaces.
- Indentation is not decoration: it decides what belongs to what.

`elif`: checked in order, first match wins

```
mark = 72
if mark >= 80:
    grade = "A"
elif mark >= 65:
    grade = "B"
elif mark >= 50:
    grade = "C"
else:
    grade = "fail"
print(grade)
```

🤔 What grade does exactly 65 get? And 64?

Name your numbers

```
TEAM_PICKUP_MIN_PORTIONS = 20      # decided by the coordinator

if portions >= TEAM_PICKUP_MIN_PORTIONS:
    ...
```

- A number inside a rule is a **decision**.
- A name lets people **find** it, **understand** it and **change** it in one place.
- CAPITAL names are **constants**: the program does not change them while it runs.

A first function: package a rule

```
def is_hot(temperature):
    if temperature >= 30:
        return True
    return False

print(is_hot(31))    # True
print(is_hot(25))    # False
```

- `def` starts the function; `temperature` is its input.
- `return` sends the answer back and **stops** the function.
- The system calls the same rule for every offer.

Test the boundary

```
def needs_team(portions):
    if portions > 20:       # rule: "20 or more"
        return True
    return False

print(19, needs_team(19))   # False  ✓
print(20, needs_team(20))   # False  ✗  should be True
print(21, needs_team(21))   # True   ✓
```

Only the **boundary** case shows the bug. Always test one below, exactly at, and one above.

⚖️ Good or risky?

Version A (from an AI assistant)

```
def label(p, a):
    if p > 20 and a == True:
        return "team"
    elif a == True:
        return "standard"
```

Version B

```
TEAM_PICKUP_MIN_PORTIONS = 20

def pickup_label(portions, approved):
    if not approved:
        return "hold"
    if portions >= TEAM_PICKUP_MIN_PORTIONS:
        return "team"
    return "standard"
```

Principle 2 · Explicit rules

- **Name your numbers**: a constant shows the decision.
- **Cover every case**: every path ends with a `return`.
- **Test the boundary**: one below, exactly at, one above.

> When you review AI code, ask: *what happens exactly at the limit, and when the condition is False?*

You build: the two pickup rules

```
TEAM_PICKUP_MIN_PORTIONS = 20

def pickup_label(portions, approved):
    # not approved → "hold"; at least the minimum → "team"
    # otherwise → "standard"
    return "?"

bridge.check("pickup_label", pickup_label,
         {"TEAM_PICKUP_MIN_PORTIONS": TEAM_PICKUP_MIN_PORTIONS})
```

- `urgency(hours_left)`: ≤ `URGENT_HOURS` → "urgent", ≤ `SOON_HOURS` → "soon", else "later".
- The check tests the boundaries **of your own numbers**.

Plug your rules into the real system

```
MY_SETTINGS = {"TEAM_PICKUP_MIN_PORTIONS": TEAM_PICKUP_MIN_PORTIONS,
               "URGENT_HOURS": URGENT_HOURS, "SOON_HOURS": SOON_HOURS}
my_system = bridge.System(MY_SETTINGS,
                          hooks={"pickup_label": pickup_label, "urgency": urgency})
my_system.run_day(show=False)
print(my_system.report())
```

- A **hook** is a place where the system lets you plug in your own function.
- If your function crashes, the system prints ⚠️ and uses its default rule.

🧭 Your decision · team size and urgency

- Change `TEAM_PICKUP_MIN_PORTIONS`, `URGENT_HOURS`, `SOON_HOURS` (or keep them and explain why).
- Rerun: the checks still pass, and the labels change.

Decision log	
My choice	team from … portions; urgent ≤ … h; soon ≤ … h
An alternative	…
Why	…
Who is affected	…

This week's tasks

- ⭐ **Core:** P1–P4, three problems in Version A, `pickup_label` and `urgency` with all ✅, plugged in, decision log.
- ⭐⭐ **Stretch:** S1 `send_team` with two conditions; `explain_label` that says why.
- 🔎 **Research 2 (5%):** a real-life rule written exactly, with a normal, a boundary and an odd case. Share it in the Week 3 warm-up.

Next week · Records and data

One offer is easy. A whole day has many. You will keep **one record per offer**, add them up, and find any offer by its **ID**.